

# C-MCPN: The Clingo McCulloch-Pitts Network Editor

An Answer-Set Programming Framework for Artificial Neural  
Networks

David Gonzalez<sup>1</sup>, Joshua T. Guerin, Ph.D.<sup>2</sup>

Mathematical Sciences

University of Northern Colorado

Greeley, CO 80639

<sup>1</sup>\*\*\*\*\*@bears.unco.edu

<sup>2</sup>\*\*\*\*\*@unco.edu

## Abstract

This is the short abstract.

## 1 Introduction

Artificial Intelligence systems have had a significant impact on the modern world over the decades. Some of the many advancements across various fields include (however certainly not limited to) conversational agents, marketing, medicine, and insurance (Nössig et al., 2024—need cite). Historically many of these advances and their impact on shaping the modern world have occurred in the background, improving our systems and applications iteratively and often at the periphery of the thinking of the general public. With new advances in AI, in particular machine learning methods (ML) and generative AI (in particular Large Language Models or LLMs), these systems have come to the forefront of public consciousness in a way that is likely to remain for the foreseeable future.

While these AI systems have the potential for significant advancement, a number of contemporary models come with significant drawbacks. In particular, the necessary growth of AI data centers and their associated infrastructure needs have caused significant stress for many local populations [Need citation]. These issues are leading to worsening availability of resources for the general public, and lead directly to the potential of significant environmental and health impacts (Bush 2025—need cite).

For continued advancement to avoid outpacing our environmental needs, significant research is necessary to bring down their respective footprints. Artificial Neural Network (ANN) technologies have made many of these technological advances possible, however drawbacks concerning their efficiency are a continued cause for concern regarding overall sustainability [cite?].

Answer Set Programming (ASP) languages often live at a different end of the research spectrum. These languages are designed to solve NP-Complete and NP-Hard problems in a way that is fast and efficient. In this paper we conduct investigate how to merge these schools of thought by building, representing, and performing inference over ANNs, specifically McCulloch-Pitts Networks (MPNs) [cite?]. To this end we introduce C-MCPN: The *Clingo McCullough-Pitts Network Editor*.

In this paper we will introduce basic techniques for modeling ANNs in the ASP language Clingo, and demonstrate how C-MCPN can be used to operate over ANN models.

## 2 Background

### 2.1 Artificial Neural Networks (ANNs)

#### 2.1.1 Efficiency and Environmental Impacts

### 2.2 Answer Set Programming (ASP)

Symbolic AI has served as a foundation for many of the earliest AI models and advancements. This form of AI relies on manipulation of symbols to represent knowledge and perform inference over models for the purpose of reasoning. These types of AI are explicitly rule-based, meaning that their decision-making systems are much more deterministic (Bhuyan et al., 2024; Liang et al., 2025; Meli et al., 2024–need cite).

The Prolog language is a classical example of these sorts of declarative (logical) programming language [cite]. Its syntax and semantics are largely derived from quantified, predicate logic, and has had a profound impact on the presentation of logic programming that continues into modern systems. Similarly, Boolean Satisfiability (SAT) solvers have served as a lower-level, contemporary form of logical programming. Throughout the 1990s and into the 2000s, these systems have expanded massively in their ability to solve large problems efficiently.

Modern ASP systems combine a simple, yet powerful syntax that is inspired by Prolog, with the efficiency of SAT systems. This empowers the user to write concise programs that can solve computationally-difficult problem instances with rather remarkable speed and efficiency.

### 2.2.1 Syntax and Semantics

In this section we will elaborate basic features of the Clingo language [cite]—a contemporary ASP language that serves as the basis for C-MCPN. In particular, we describe the nature of *facts*, *rules*, and *cardinality constraints* that are used in our representations<sup>1</sup>.

The most atomic units in Clingo are *facts*, presented as predicate-based statements. The basic syntax is in the form `property(atom)`, which ascribes a property, `property`, to a symbol `atom`.

E.g.,

```
red(apple).
```

is a complete statement (and program) in Clingo, attributing a property of `red` to an atom named `apple`. You will note that in this, and future examples, our code operates in a declarative, and non-algorithmic context. Rather than providing discrete instructions to the processor regarding specific steps to solve a problem, these logical languages allow us to *describe* the nature of the world and the problem that we wish to solve, as well as what a solution *looks like* in our domain. It is the role of the *solver* to find a solution that matches our description.

Further knowledge can be *inferred* through combinations of predicates that create *rules* of the form: `consequent :- antecedent`, mirroring the *implication* from formal logic (e.g.,  $\text{consequent} \leftarrow \text{antecedent}$ ). Such rules specify that the *truth* of the `consequent` follows logically from the truth of the `antecedent`.

For example,

```
eat(apple) :- red(apple), ripe(apple).
```

describes a preference over what fruit will be eaten. Specifically, the modeled preference indicates that “if an apple is red and it is also ripe, then we eat the apple.”

More generally, we often phrase rules around *variables* for a greater degree of generality in our software. In this way we can describe the nature of entire sets of items, rather than specific atoms. These are identified by an uppercase starting symbol, and are treated as *patterns* to be matched by the system (i.e., variables can match specific atoms based on context).

I.e.,

```
eat(Fruit) :- red(Fruit), ripe(Fruit).
```

---

<sup>1</sup>A full review of the ASP family’s syntax and semantics is outside the scope of this work, we seek to provide minimal instruction. For users with previous experience in Prolog or a similar logical dialect, the syntactic roots of ASP languages should be at somewhat familiar.

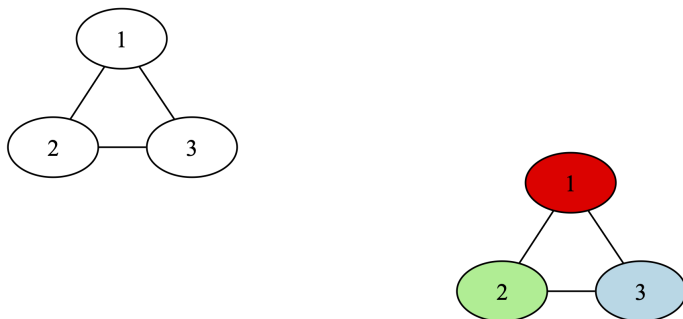
In this case, `Fruit` would match any contextually-relevant symbol, like the atom `apple` that we introduced earlier. In this case the use of variables could extend the notions of edibility, redness, and ripeness to other possible fruits.

### 2.2.2 Problem Modeling in Clingo: 3-Colorability

As an illustrative example regarding how to model problems in Clingo, we will use the graph 3-Colorability problem, as it is presented in *ASP $\forall$ : Answer Set Programming "Algorithms"* (Guerin et al., 2026–need cite). ASP $\forall$  is an Open Educational Resource (OER) for answer set programming in the Clingo dialect. These implementations provide minimal examples of how to model problems in Clingo, and graph-based implementations are of particular interest to us as ANNs are also graphical in nature.

The 3-Colorability problem is deceptively easy to state, but is computationally expensive to solve exactly. A simplified statement of the problems is “Given a graph,  $G$ , assign one of 3 colors to the vertices of  $G$  such that no two adjacent vertices share a color.”

E.g., Consider the following graph and proposed solution:



*Need to fix formatting.*

In this particular case, color assignment is rather trivial—any assignment of all three colors to the vertices will lead to a valid coloring.

The implementation for 3-Colorability provided by ASP $\forall$  is specified in terms of an *instance* as well as a *solution* file. The instance encodes input information in the form of predicates<sup>2</sup>, and the solution provides a general implementation for inference over any provided instance model.

---

<sup>2</sup>Unlike many programming language paradigms, ASP languages often lack traditional IO systems, moving these features to dedicated parsers in other languages.

```

node(1..3).
node(1, 2).
node(1, 3).
node(2, 3).

% Set of available colors.
color(1..3).

% Edges are symmetric.
% Assumes potentially directed description of an undirected graph.
edge(M, N) :- edge(N, M).

% Each vertex is assigned a single color.
1 { color(N, C) : color(C) } 1 :- node(N).

% No two adjacent vertices can have the same color.
:- edge(N,M), color(N, C), color(M, C).

% Output color information.
#show color/2.

clingo version 5.6.2

Reading from clingo_test.lp

Solving...

Answer: 1

color(1,2) color(2,3) color(3,1)

SATISFIABLE

Models : 1+
Calls : 1
Time : 0.013s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time : 0.002s

```